

# Lecture 3: Complexity, Real-World Performance & Profiling

## Drilling down into performance

## Experimental vs Theoretical

## Complexity

## Formal analysis

- We can use the tools of formal algorithmic analysis to understand running time
  - ...and memory use, too
- This is Real Computer Science(tm) but also practical:
  - What algorithm should we choose for different size graphs?
  - Can we tweak algorithms for special forms of graphs to be faster?
- Formal tool to compare algorithms: Big-  $O$  /  $\Omega$  /  $\Theta$  Notation

## Big- $O$ / $\Omega$ / $\Theta$ Bounds

- This notation establishes (asymptotic) *bounds* for runtime/memory use of algorithms

- At the lower end, these are heavily machine-dependent
- In the high end, they tend asymptotically towards their bounds
  - This is *worst-case analysis*

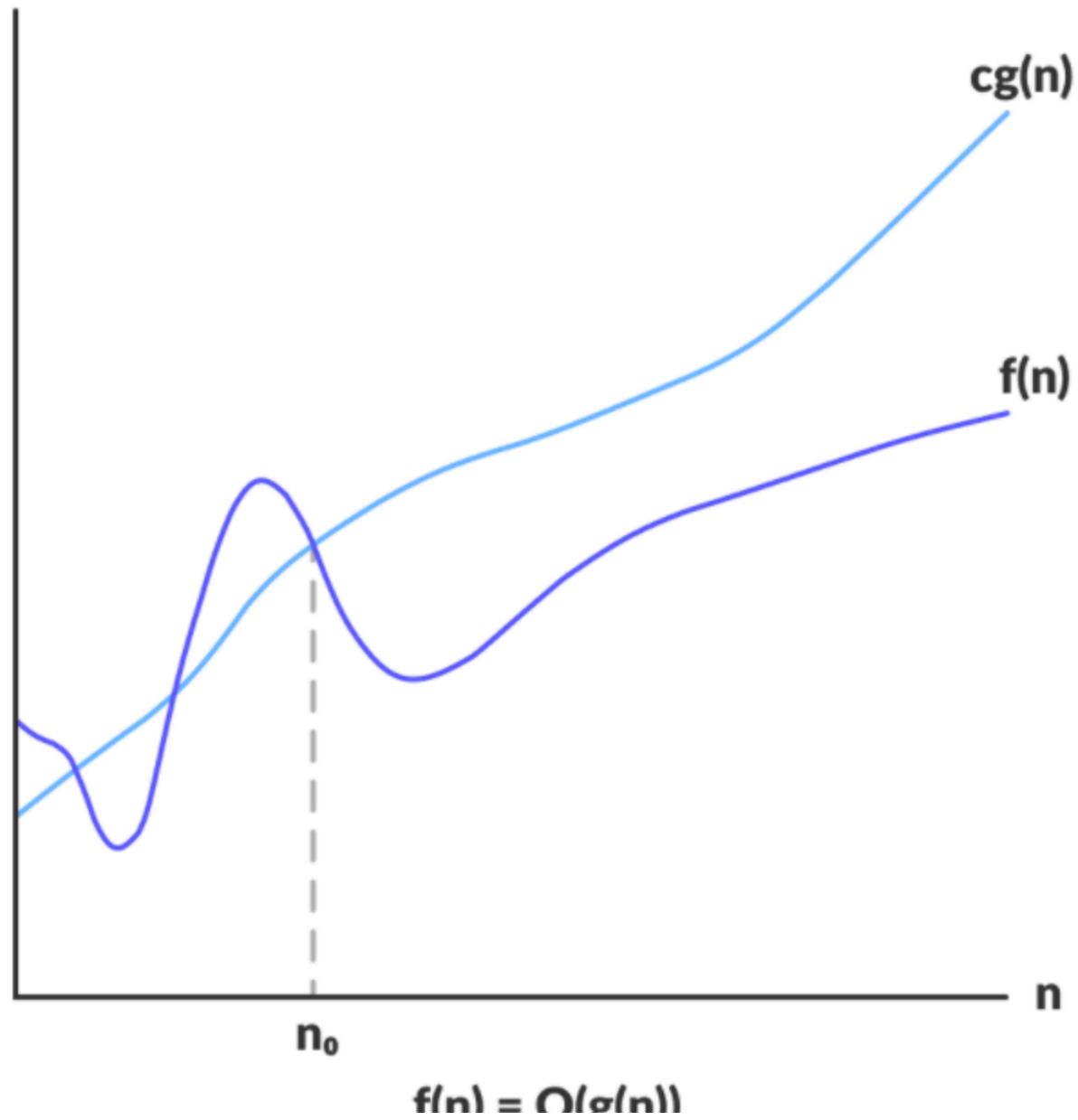
## Definitions (yay!)

- We define an appropriate function  $g(n)$  and constant  $c$  so that for any  $n > n_o$ :
- Upper bound:  $f(n) = O(g(n)) : f(n) \leq c \cdot g(n)$
- Lower bound:  $f(n) = \Omega(g(n)) : f(n) \geq c \cdot g(n)$
- Fit bounds:  $f(n) = \Theta(g(n)) : c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

## A picture is worth...

**...about 3 definitions**







## How to find upper bounds (heuristic!)

1. Take an algorithm
2. Write down what happens
  - Eg. "loop through every vertex in the graph and add its degrees to sum"
3. Write down how often it happens *in the worst case*
  - E.g. "for every vertex"
4. Sum up the sizes of every step
5. Pick the largest polynomial term

## Weighted graphs

- We've (casually) used the word "distance" so far for each edge as if it is "1"
- A natural extension is to *weigh* the edges to indicate distances  $> 1$ 
  - Or model any other factor between vertices, e.g. capacity, magnitude etc.
  - Can be negative as well, though that plays havoc with distances!

Def.3.6: A *weighted graph*  $G$  is a graph for which each edge  $e$  has an associated real-valued number  $w(e)$  called its weight. For any subgraph  $H \subseteq G$ , the weight  $w(H) = \sum_{e \in E(H)} w(e)$

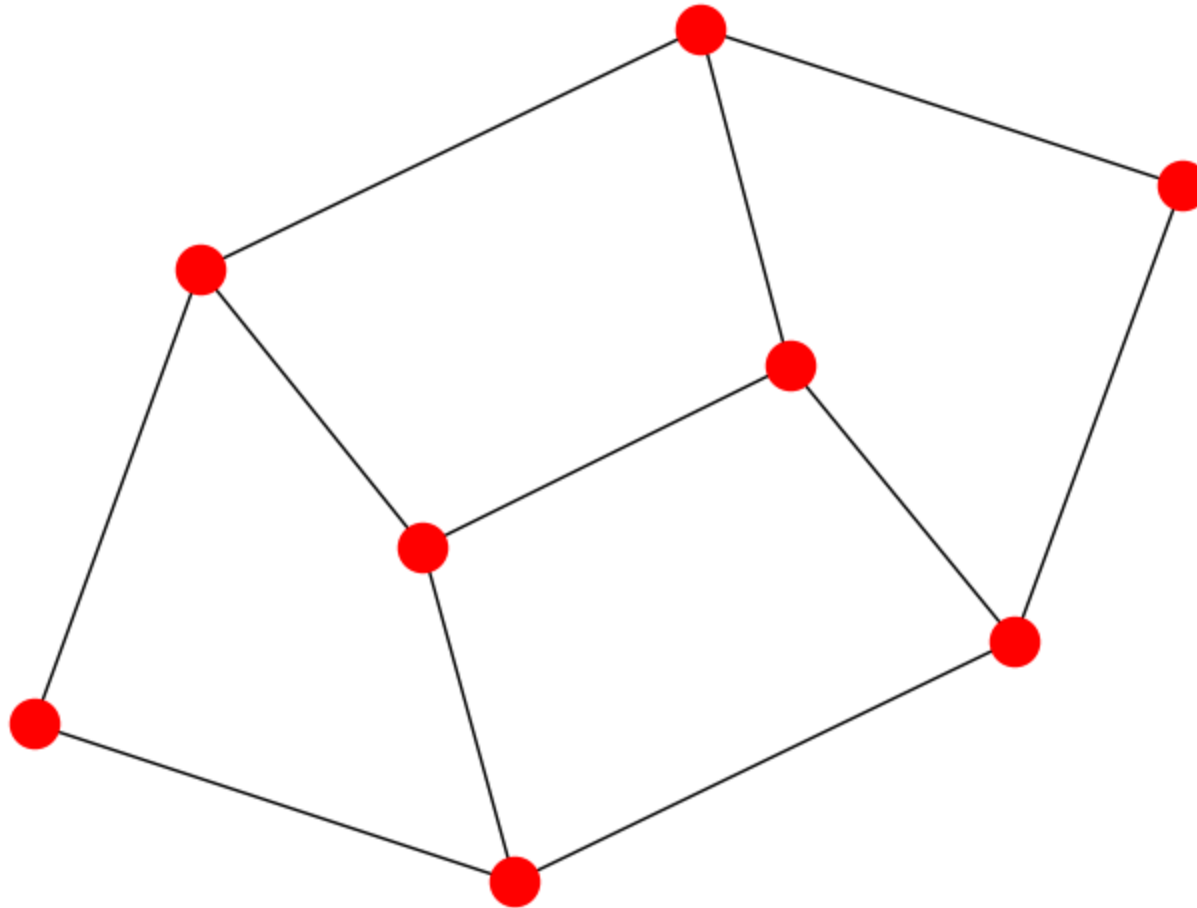
## Dijkstra's Shortest Path Algorithm

- Find shortest paths from a vertex  $v$  to every other vertex

- Uses weights as proxy for distance and *minimizes* them
  - So be careful what you model this way!
- We are going to skip the formal description of the model (Algorithm 3.2 GTCN67-68)
- We are going to use our favorite "pseudo"code (Note 3.5) – Python Code!

## Dijkstra's Algorithm

- I've set up (hidden) Python NetworkX graph G in the shape of fig 3.4 and initialized it with 0 distance for node 0 and "Inf" elsewhere





## Dijkstra's Algorithm

```
def Dijkstra(G, u):
    S_u = set([u])
    V_G = set(G.nodes())
    G._ = G.nodes
    while S_u != V_G:
        R_u = S_u | set([i for v in S_u for i in G[v]])
        for y in R_u - S_u:
            for x in set(G[y]) & S_u:
                if G._[x]['L2'] + G[x][y]['w'] < G._[y]['L2']:
                    G._[y]['L1'] = x
                    G._[y]['L2'] = G._[x]['L2'] + G[x][y]['w']
            min_v = min(V_G - S_u, key = lambda v: G._[v]['L2'])
            S_u.add(min_v)
    return S_u
```

[Copy](#)

## Dijkstra's Output

```
S_u = Dijkstra(G, 0)

for v in S_u:
    print(v, G._[v]['L2'])
```

[Copy](#)

## Measuring Dijkstra's $O(n)$

- We can use an *experimental* approach
- Use the python: `timeit` module function 'timeit'
- NB: don't forget to include the "globals=globals()" parameter to access your own code!

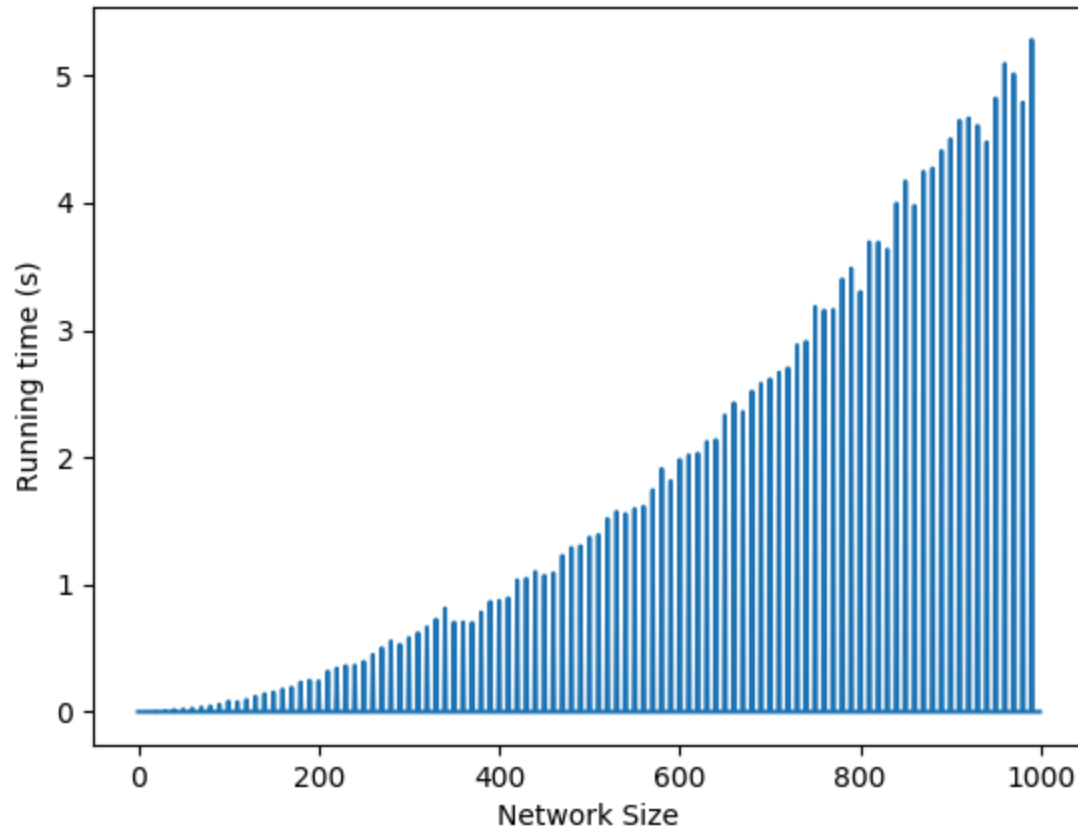
```
from timeit import timeit
def testG(G):
    for v in G.nodes:
        Dijkstra(G,v)

print("Time [ms]:", timeit('testG(G)', setup='', globals=globals(), number=1))
```

[Copy](#)

```
Time [ms]: 0.0036798836663365364
```

## Drawing asymptotic complexity



## Real-world performance

- We execute code on a very complicated stack of technologies
- CPU -> OS -> Python interpreter -> libraries -> program
- Much of what we work with we can't change
- algorithm choice matters (big O!)
- library choice matters

program's speed is affected by every layer in this stack: The complexity comes largely from how Python operates. Python code is translated into bytecode, and the Python interpreter then acts as a virtual machine (VM) to run that bytecode. This layer of translation is why running code on the VM is generally slower than running native machine code.

| Layer        | Component                                        |
|--------------|--------------------------------------------------|
| Hardware     | Central Processing Unit (CPU)                    |
| Software     | Operating System (OS)                            |
| Interpreter  | Python Interpreter (acting as a Virtual Machine) |
| Dependencies | External Libraries                               |
| Your Code    | The specific program                             |

## CPU: Units working with a clock

# Central Processing Unit (CPU)

- Consists of the
  - Control Unit
  - Arithmetic Logic Unit (ALU)
  - Registers

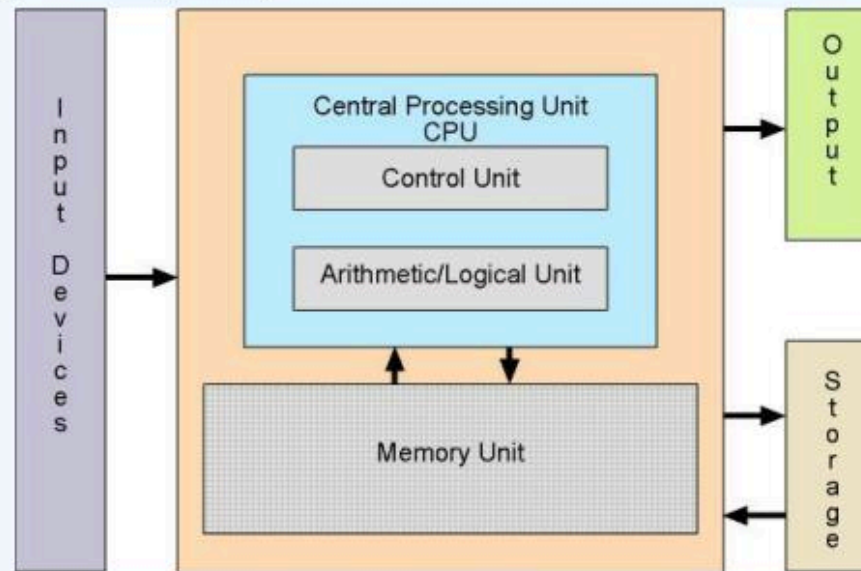
### RAM VS. REGISTERS

#### Registers:

These are small, extremely high-speed storage locations located inside the CPU. They hold data and instructions that the Arithmetic Logic Unit (ALU) and Control Unit are currently processing. They are the fastest memory you can access.

#### Main Memory (RAM)- Memory Unit:

This is the large pool of memory that holds the running programs and data (like Python code's instructions and variables). is connected to the CPU but is physically separate.



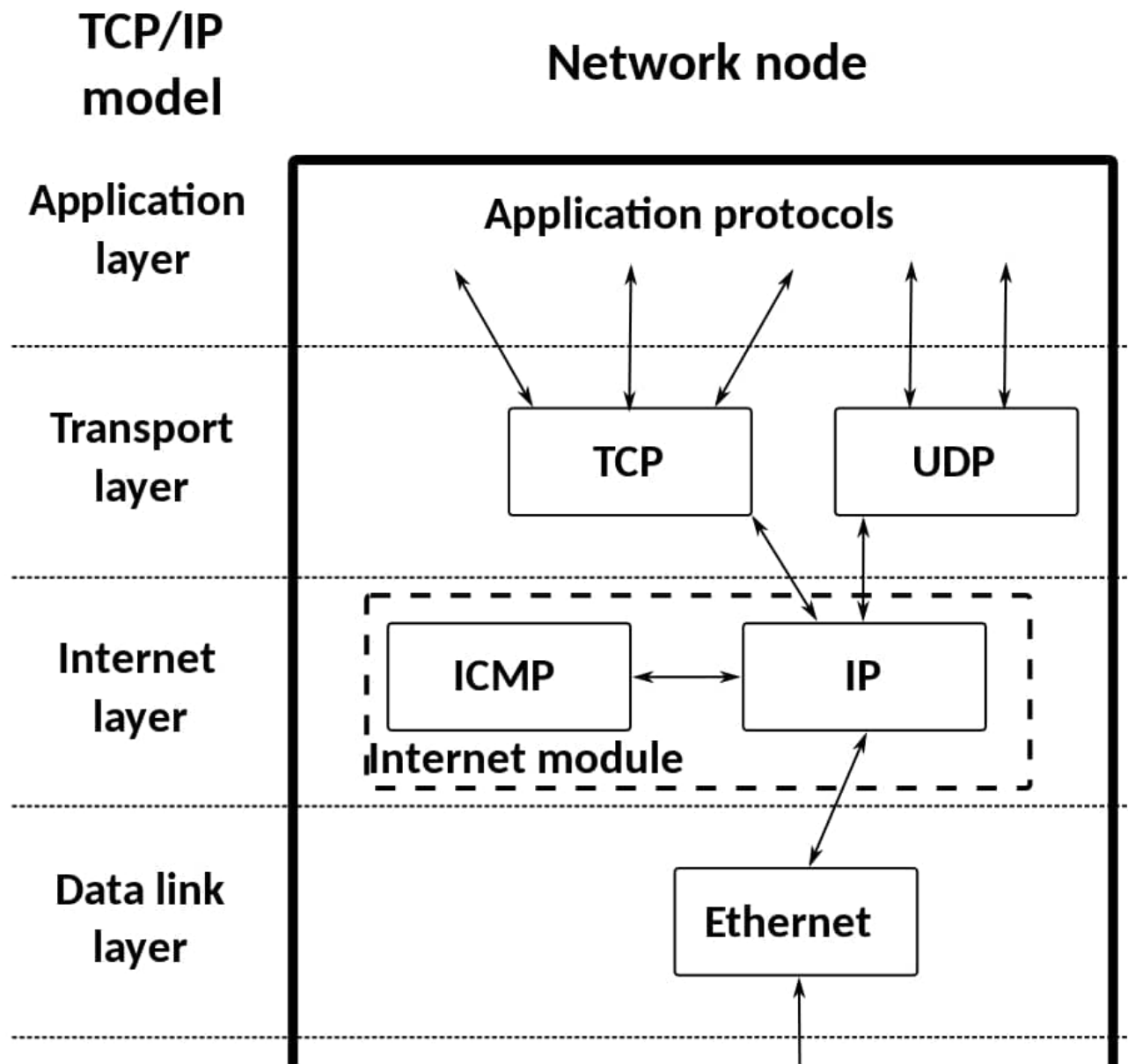
## Python as a virtual machine

- Python code is translated into bytecode
- Python interpreter acts like a virtual machine
- translation is done once (on source change); ".pyo", ".pyc" files
- use the `dis` module to see it!
- Running on the VM is slower
- Many libraries (e.g. OpenMPI, numpy) translate functions to C/C++ code

## Very basic **networking**

| <b>The TCP/IP model provides a standardized structure for organizing how network communication works, breaking down the complex task into four main layers .</b> |                                                                                  |                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|-----------------------------------------|
| Layer                                                                                                                                                            | Function                                                                         | Key Protocols Mentioned in Diagram      |
| 1. Application layer                                                                                                                                             | Provides services directly to the user's application.                            | Application protocols (e.g., HTTP, DNS) |
| 2. Transport layer                                                                                                                                               | Manages end-to-end communication and data delivery.                              | TCP, UDP                                |
| 3. Internet layer                                                                                                                                                | Handles addressing and routing of data packets across different networks.        | IP, ICMP                                |
| 4. Data link layer                                                                                                                                               | Manages physical transmission of data between devices on the same local network. | Ethernet                                |







## Network cable

**DeadLock:** In this scenario, Process 1 cannot proceed until it gets Resource B, which is held by Process 2. Process 2 cannot proceed until it gets Resource A, which is held by Process 1.

## Synchronization...

| Process   | Resource Currently Holding | Resource Currently Waiting For |
|-----------|----------------------------|--------------------------------|
| Process 1 | Resource A                 | Resource B                     |
| Process 2 | Resource B                 | Resource A                     |

- resource contention with shared datastructures rises with number of workers
- queue's (think: `MPI.COMM.world!`)
- deadlock: nobody gets to do anything!
- solution: timed cooldowns

## ...over the network

- the latency is very much increased, leading to quicker deadlock
- when shuttling significant information around, bandwidth degrades too

## Profiling Experimentally

Unix `time`

## The python profiler

- Looks into your code
- built-in

## What does the profiler output mean?

- time spent in your code vs libraries
- function calls
- optimization hooks

## How to identify & fix hotspots

- check what takes the most time using profiling
- is it your program, or a library?
- can you improve the algorithm
- can you make fewer function calls?

```
import cProfile
import time
import re
def process_data(data_size):
    """A function that simulates a time-consuming data process."""
    result = 0
    for i in range(data_size):
        # This calculation simulates a CPU-heavy task
        result += i * i**2 * 5
    return result
def main_bottleneck_test():
    # PART 1: The Bottleneck (CPU-heavy)
    # This will be the "hotspot" we look for in the profile output.
    process_data(50000)

    # PART 2: I/O-heavy (Waiting time)
    # This simulates waiting for a file or network, which is time-
    consuming but not CPU-intensive
    time.sleep(0.01)

    # PART 3: Quick task
    re.match(r'a', 'abcdef')
# Use cProfile.run() to profile the main function
print("--- Profiler Output ---")
cProfile.run('main_bottleneck_test()')
```